Virtual Function

Definition:
A **virtual function** is a member function of a class (in object-oriented programming, especially **C++**) that is declared using the keyword virtual and is meant to be **overridden in a derived (child) class**. The function call is **resolved at runtime** based on the type of object being pointed to, not the type of pointer.

Why Virtual Functions Are Needed

Virtual functions support **runtime polymorphism**. This allows a base class pointer to call the correct derived class function when it points to a derived class object.

Without virtual functions, function calls are resolved at **compile time**, which can lead to incorrect behavior when inheritance is used.

How Virtual Functions Work

- A function is declared virtual in the **base class**.
- A derived class **overrides** this function.
- When a base class pointer points to a derived class object and calls the function:
    - The **derived class's version** is executed.
- This decision is made **at runtime** (dynamic binding).

Key Points About Virtual Functions

- Declared using the keyword **virtual**
- Enable **dynamic (late) binding**
- Used to achieve **runtime polymorphism**
- Function call depends on the **object type**, not pointer type
- Usually defined in the **base class**
- Often used in **inheritance hierarchies**

Advantages

- Ensures correct function calls in inheritance
- Makes programs more flexible and extensible
- Supports polymorphic behavior

Disadvantages

- Slightly slower than non-virtual functions due to runtime lookup
- Uses extra memory (via virtual table – *vtable*)

Conclusion

A virtual function ensures that the **correct function is called for an object**, even when accessed through a base class pointer, making it a key concept in **object-oriented programming and polymorphism**.

Example:

```
class A {
public:
   virtual void show() {
      cout << "Class A";  }
};
class B : public A {
public:
   void show() {
      cout << "Class B";    }
};
A* obj = new B();
obj->show();
```

OUTPUT:

Class B

In Simple Words

- Pointer is of **A**
- Object is of **B**
- Because show() is **virtual**, **B's function runs**

✓ Without virtual → A's function runs
✓ With virtual → B's function runs

<u>Identifier:</u>

Definition:
An **identifier** is the **name given to a variable, function, class, array, or any other user-defined item** in a program to identify it.

Rules for Identifiers

1. Can contain **letters (a–z, A–Z), digits (0–9), and underscore (_)**
2. **Must start with a letter or underscore**, not a digit
3. **No spaces** allowed
4. **Cannot be a keyword** (like int, return, if)
5. Are **case-sensitive** (Age and age are different)

Valid Identifiers

- sum
- total_marks
- _count

Invalid Identifiers

- 1sum ✕ (starts with number)
- total marks ✕ (space)
- int ✕ (keyword)

Why Identifiers Are Used

- To **store values**
- To **access data easily**
- To **make programs readable**

Example:

float price;

int sum;

void show();

Identifiers are:

- price
- sum
- show

<u>Operators:</u>

Operators are **symbols** used to **perform operations** on variables and values.

Examples

int a = 10, b = 5;

Here:

- + , - , * , / , % are operators

Types:

1. Arithmatic
2. Logical
3. Rational
4. Assignment

1. Arithmetic Operators

Used for **mathematical calculations**.

Operator	Use	Example
+	Addition	`a + b`
-	Subtraction	`a - b`
*	Multiplication	`a * b`
/	Division	`a / b`
%	Remainder	`a % b`

Example:

```
int a = 10, b = 5;
```

```
int sum = a + b;
```

2. Logical Operators

Used to **combine conditions**.

Operator	Meaning
&&	AND
`	
!	NOT

Example:

```
if (a > 5 && b < 10)
```

4. Relational Operators

Used to **compare two values** and give **true or false**.

Operator	Meaning
>	Greater than
<	Less than
==	Equal to
!=	Not equal

Example:

if (a == b)

3. Assignment Operators

Used to **assign values** to variables.

Operator	Example
=	a = 10
+=	a += 5
-=	a -= 2

Dynamic Memory Allocation:

Dynamic Memory Allocation means **allocating memory during program execution (runtime)** instead of before the program starts.

👉 Memory is taken **only when needed** and can be **released after use**.

Why It Is Used

- When we **don't know the size of data in advance**
- To **use memory efficiently**

Dynamic Memory Allocation in C++

C++ uses:

- new → to allocate memory
- delete → to free memory

Real-Life Example

Think of booking seats:

- Fixed memory → booking fixed seats in advance
- Dynamic memory → booking seats **only when people arrive**

Why Dynamic Memory Allocation Is Important

- Saves **memory space**
- Useful for **large data**
- Memory can be **freed and reused**

Short Example

```
int* num = new int;   // memory created

*num = 20;

cout << *num;

delete num;         // memory released
```

Dynamic vs Static Memory

Static	Dynamic
Fixed size	Flexible size
Memory at compile time	Memory at runtime
Cannot be resized	Can be resized

Uses of Dynamic Memory Allocation

1. Used when **memory size is not known in advance**
2. Helps to **save memory**
3. Useful for **large data and arrays**
4. Memory can be **released after use**
5. Makes programs **flexible and efficient**

Keywords Used in C++

Keyword	Purpose
new	Allocates memory
delete	Frees memory
new[]	Allocates array memory
delete[]	Frees array memory